

Name :- Mihir Mungara

Snowflake Assignment-7 (Security and Access Management in Snowflake)

Created and Managed Users and Roles :-

```
-- Step 1: Create Roles
CREATE ROLE admin;
CREATE ROLE analyst;
CREATE ROLE viewer;

-- Step 2: Assign Privileges to Roles
grant all privileges on database our_first_db to role admin;
grant select on all tables in schema our_first_db.public to role analyst;
grant usage on warehouse compute_wh to role viewer;

-- Step 3: Create Users
CREATE USER john_doe
PASSWORD = 'Password123!'
DEFAULT_ROLE = analyst
MUST_CHANGE_PASSWORD = TRUE;

CREATE USER admin_user
PASSWORD = 'Password123!'
DEFAULT_ROLE = admin
MUST_CHANGE_PASSWORD = TRUE;

CREATE USER jane_doe
PASSWORD = 'Password123!'
DEFAULT_ROLE = viewer
MUST_CHANGE_PASSWORD = TRUE;

-- Step 4: Assign Roles to Users
GRANT ROLE admin TO USER admin_user;
GRANT ROLE analyst TO USER john_doe;
GRANT ROLE viewer TO USER jane_doe;

GRANT ROLE admin TO USER mihir;
GRANT ROLE analyst TO USER mihir;

show users;
```

Configured Network Policies :-

```
-- 2. Configure Network Policies

-- Step 1: Create a Network Policy
CREATE NETWORK POLICY secure_policy
ALLOWED_IP_LIST = ('192.168.1.0/24', '203.0.113.0/32')
BLOCKED_IP_LIST = ('10.0.0.0/8');

-- Step 2: Apply Network Policy
ALTER ACCOUNT SET NETWORK_POLICY = secure_policy;
```

Implemented Data Masking :-

```
-- 3. Implement Data Masking on Sensitive Data
-- Step 1: Create a Table with Sensitive Data
CREATE or replace TABLE employees (
    id INT,
    name STRING,
    ssn STRING
);
INSERT INTO employees VALUES (1, 'Alice', '123-45-6789');

-- Step 2: Define a Masking Policy
CREATE MASKING POLICY ssn_mask_updated AS (val STRING) RETURNS STRING ->
CASE
    WHEN CURRENT_ROLE() = 'ADMIN' THEN val
    ELSE 'XXX-XX-XXXX'
END;

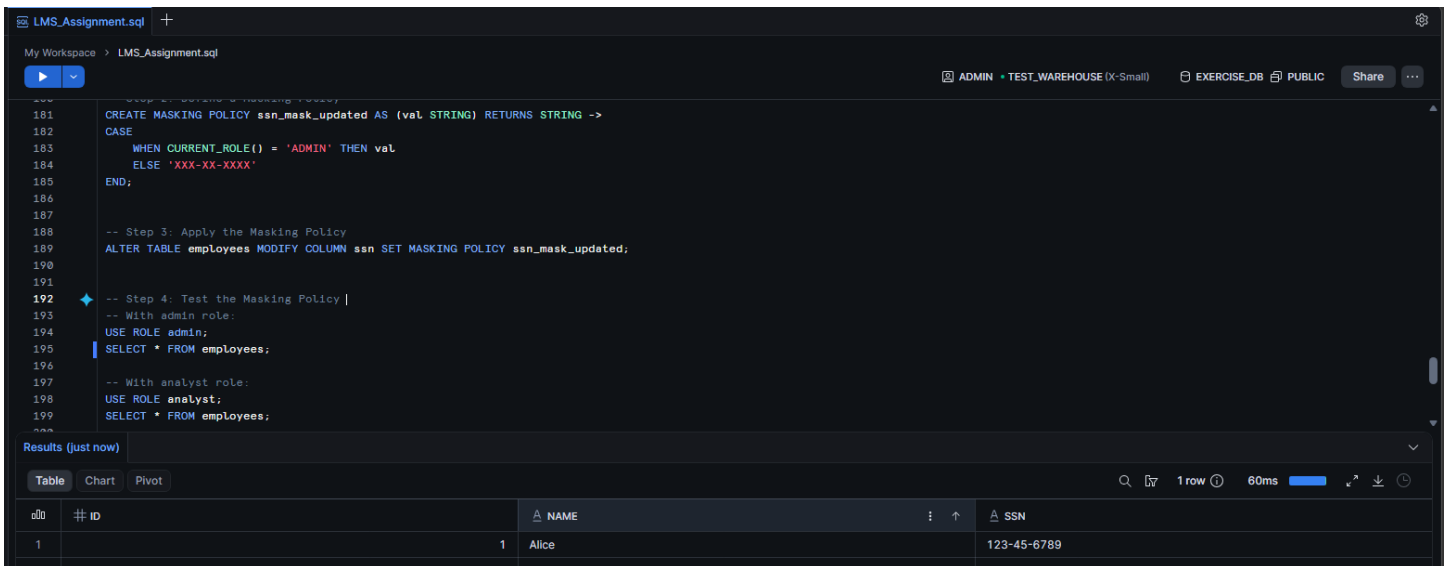
-- Step 3: Apply the Masking Policy
ALTER TABLE employees MODIFY COLUMN ssn SET MASKING POLICY ssn_mask_updated;

-- Step 4: Test the Masking Policy
-- With admin role:
USE ROLE admin;
SELECT * FROM employees;

-- With analyst role:
USE ROLE analyst;
SELECT * FROM employees;

use role accountadmin;
```

Used Role Admin then Unmasked :-



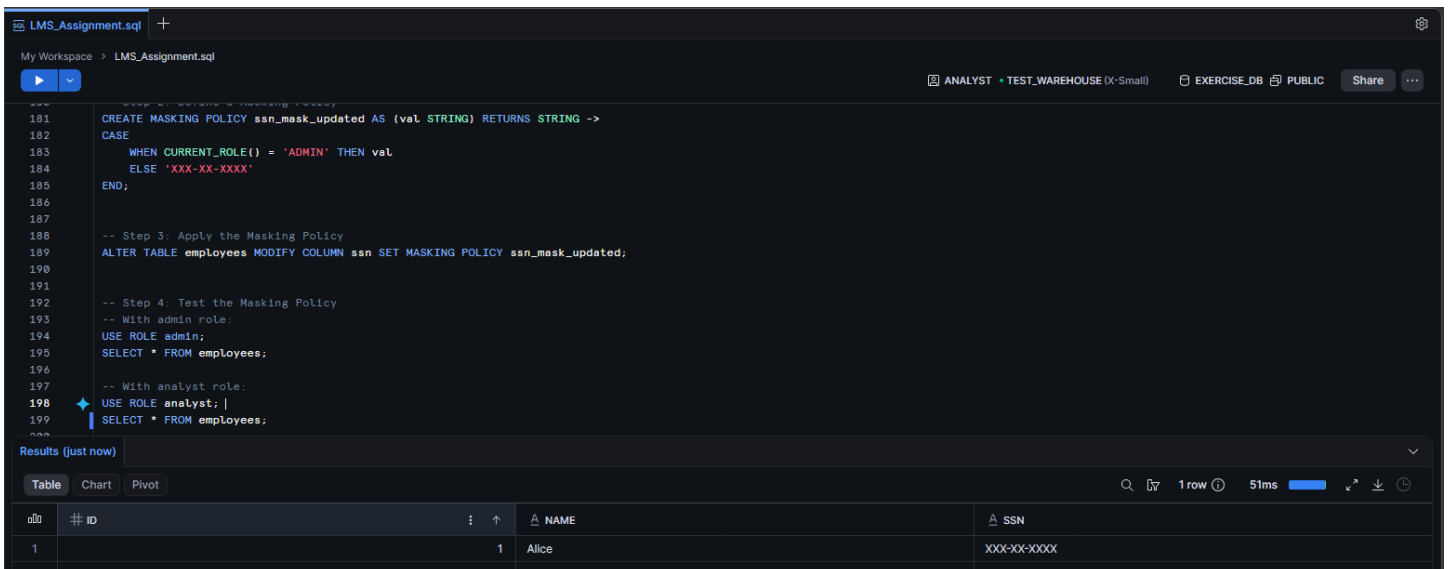
The screenshot shows a SQL IDE with a script named `LMS_Assignment.sql`. The script defines a masking policy `ssn_mask_updated` that returns the original SSN for the `ADMIN` role and a masked value for other roles. It then applies this policy to the `ssn` column of the `employees` table. The execution is performed using the `ADMIN` role, and the results show the unmasked SSN for the employee with ID 1.

```
181 CREATE MASKING POLICY ssn_mask_updated AS (val STRING) RETURNS STRING ->
182 CASE
183   WHEN CURRENT_ROLE() = 'ADMIN' THEN val
184   ELSE 'XXX-XX-XXXX'
185 END;
186
187
188 -- Step 3: Apply the Masking Policy
189 ALTER TABLE employees MODIFY COLUMN ssn SET MASKING POLICY ssn_mask_updated;
190
191
192 -- Step 4: Test the Masking Policy |
193 -- With admin role:
194 USE ROLE admin;
195 SELECT * FROM employees;
196
197 -- With analyst role:
198 USE ROLE analyst;
199 SELECT * FROM employees;
```

Results (just now)

#	ID	NAME	SSN
1	1	Alice	123-45-6789

When Used Analyst then Data is Masked :-



The screenshot shows the same SQL IDE with the same script. This time, the execution is performed using the `ANALYST` role. The results show the masked SSN for the employee with ID 1.

```
181 CREATE MASKING POLICY ssn_mask_updated AS (val STRING) RETURNS STRING ->
182 CASE
183   WHEN CURRENT_ROLE() = 'ADMIN' THEN val
184   ELSE 'XXX-XX-XXXX'
185 END;
186
187
188 -- Step 3: Apply the Masking Policy
189 ALTER TABLE employees MODIFY COLUMN ssn SET MASKING POLICY ssn_mask_updated;
190
191
192 -- Step 4: Test the Masking Policy
193 -- With admin role:
194 USE ROLE admin;
195 SELECT * FROM employees;
196
197 -- With analyst role:
198 USE ROLE analyst; |
199 SELECT * FROM employees;
```

Results (just now)

#	ID	NAME	SSN
1	1	Alice	XXX-XX-XXXX